Simple Task Scheduler

Abara, D.

July, 2026

Introduction

This document provides background information about the simple bare-metal task scheduler that toggles four LEDs. It uses the internal Systick Timer/interrupt to switch between tasks and is not meant to be a performant example but rather a demonstration of how such an application can be built, and for learning. The hardware used is the STM32F401RE-Nucleo board from ST-Micro with 96kb of RAM. A chart of the program flow is given in Figure 1. Some key processes from the flowchart are explained in the following sections.

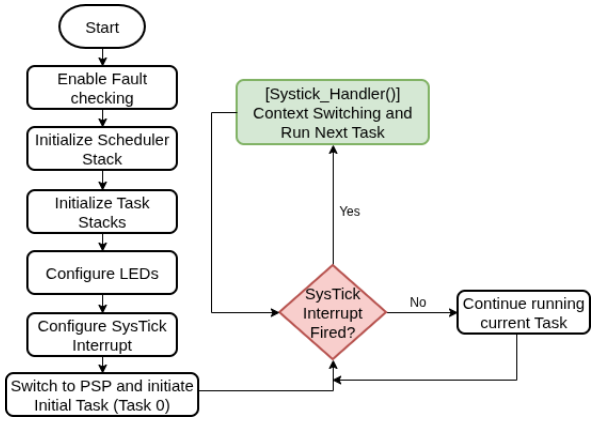

Figure 1: Program flowchart

Intialization of Scheduler & Task Stacks

The MCU stack is split into two segments, one for the user processes or tasks to be run tracked using the Processor Stack Pointer (PSP) and one for the MCU processes tracked uisng the Main Stack Pointer (MSP). Each user task is allocated a stack size of 1KB beginning from the end of stack `SRAM_END`. The CPU stack is also allocated 1KB. Figure 2(a) depicts this allocation noting that RAM starts at `0x20000000` for the chosen MCU. To initialize the user stacks, the required registers (stack frames) are shown in Figure 2(b). Stack Frame 1 (SF1) is normally pushed to stack by the processor when an exception occurs while Stack Frame 2 (SF2) is handled by the programmer. However, in the initial case, all of these addresses are empty or hold gabbage values hence we need to initialize them with proper values.

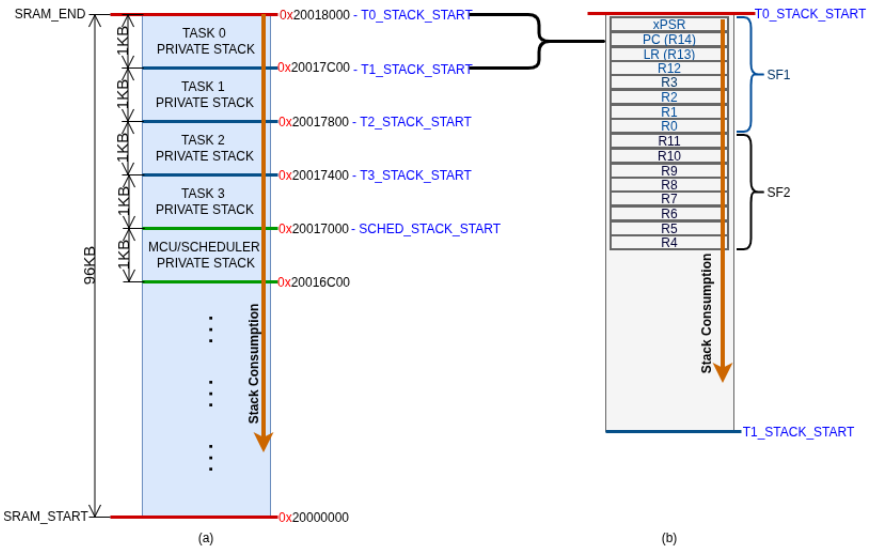

Figure 2: (a) MCU stack split (b) T0 private stack showing how stack frames are stored for context switch

For each private stack:

- The `xPSR` is initialized with `0x01000000U`. This is to set the Tbit to 1 for Thumb instruction set architecture.
- The Program Counter is set to the address of the task handler for the specific task. For example, for task 0, PC is set to the address of `task0_handler`.
- The Link Register is set to a special `EXC_RETURN` (exception return) value of `0xFFFFFFFD` which is to return back to PSP.
- The remaining registers are all initialized to zero.

After the first initialization, the processor is responsible for pushing and popping SF1 for each of the user task stacks, while we will implement the pushing and popping of SF2 for context switching.

LED configuration

For this application, four LEDs are used - the internal LED of the STM32F401RE, and three external LEDs all connected to the MCU's PortA (PA5-PA8) on bus AHB1. A depiction of the external circuit connection is given in Figure 3.

Set SP to use Processor Stack Pointer

By default, the processor starts with the MSP but we want only the processor core operations to use MSP while the user tasks of switching the LEDs use their own separate private tasks as depicted in

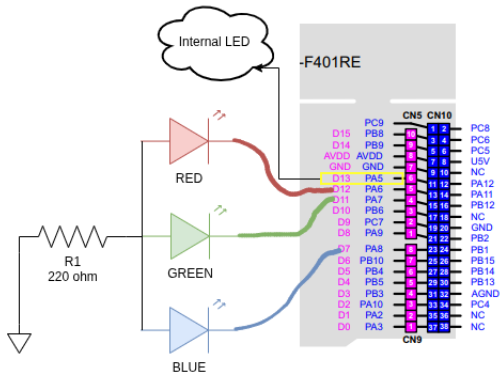

Figure 3: External LED circuit

Figure 2(a). For this reason, before starting the user tasks, we want to change from MSP to PSP and make sure PSP holds the address of the task to be run. In the first case, the task to run is Task 0 so after switching to PSP, the initial task (Task 0) is then started.

Systick Interrupt & Systick_Handler

To keep things simple, the application uses the Systick timer to control the intervals for switching from one task to the other. The internal clock of the MCU is 16MHz and the timer is configured to interrupt the processor every 1 millisecond by default. After every 1 millisecond (or 1000 Hz or any other user-configured time period), the Systick interrupt will be triggered and the `Systick_Handler` will be called to perform the context switch. In other words, the `Systick_Handler` functions as the scheduler. Although this may not be the best practice, note that this project was completed as a way to deeply understand the concept of context switching at the register level and is not meant to be a performant example. Within the `Systick_Handler`, the following process is completed:

- Push the context of the current task to that task's private stack. For example, if task0 is running, we push SF2 of task0 to task0's stack.
- Save the final psp value of the current task. This is important so that when we want to run this task, we know exaclty where in memory we need to start popping values from.
- Decide on the next task to run, for example, if task0 is currently running, then we want to run task1. So we need to decide this programmatically.
- Retrieve the psp value of the next task to be run. We need this value so that we can retrieve the context of the next task from its private stack. For example, if task1 is the next task, then we need the last psp value after its context was saved, so that we can use that to retrive task1's context.
- Retrive the context of the next task from its private stack and then run the task.